VajraKavach

Secure Job Search & Professional Networking Platform

Project Milestone 1 Report

CSE 345/545: Foundations to Computer Security

Team Members	Roll Number
Yash Kumar Vaibhav	MT25091
Aayush Gupta	MT25008
Ayush Jain	MT25066
Ayush Kumar	2020290

Indraprastha Institute of Information Technology, Delhi

February 13, 2026

Contents

1 Technology Stack **2**
1.1 Backend . . . 2
1.2 Frontend . . . 2
1.3 Infrastructure . . . 2
1.4 Technology Stack Summary . . . 3

2 HTTPS Configuration with Self-Signed Certificates **4**
2.1 Vite Configuration . . . 4
2.2 How It Works . . . 4
2.3 Screenshots . . . 5

3 Skeleton Application Deployment on VM **6**
3.1 Project Structure . . . 6
3.2 Startup Process . . . 6
3.3 Screenshots . . . 7

4 Summary **9**

Technology Stack

After careful evaluation of the course requirements and recommended tools, we finalized the following technology stack for VajraKavach:

Backend

- **Language:** Python 3.10
- **Framework:** FastAPI 0.109.2 — high-performance async web framework with automatic OpenAPI documentation
- **ASGI Server:** Uvicorn 0.27.1 with standard extras for production-grade serving
- **ORM:** SQLAlchemy 2.0.27 (async mode via `asyncpg` driver)
- **Database:** PostgreSQL 14 (running on localhost:5432)
- **Authentication:** python-jose (JWT HS256), passlib with Argon2id hashing
- **TOTP:** PyOTP 2.9.0 with QR code generation (qrcode + Pillow)
- **Encryption:** cryptography 42.0.5 (Fernet AES-128-CBC for data-at-rest, RSA-2048 for PKI)

Frontend

- **Library:** React 18.2
- **Build Tool:** Vite 5.4.21 (with HMR for rapid development)
- **Styling:** Tailwind CSS 3.4
- **Routing:** React Router 6.22
- **HTTP Client:** Axios 1.6
- **Icons:** Lucide React
- **HTTPS:** `@vitejs/plugin-basic-ssl` for self-signed TLS certificates

Infrastructure

- **Operating System:** Ubuntu (IIITD VM)
- **Database Server:** PostgreSQL 14
- **Process Management:** Custom `start.sh` / `stop.sh` scripts
- **Version Control:** Git + GitHub (`github.com/yashkumarvaibhav/VajraKavach`)

Technology Stack Summary

Component	Technology
Backend Framework	FastAPI (Python 3.10)
Frontend Framework	React 18 + Vite 5
Database	PostgreSQL 14
Authentication	JWT (HS256) + Argon2id + TOTP
Encryption	Fernet AES (data-at-rest), RSA-2048 (PKI)
HTTPS	Self-signed TLS via Vite basicSsl plugin
Styling	Tailwind CSS 3.4
OS	Ubuntu (VM)

Table 1: Technology Stack Overview

HTTPS Configuration with Self-Signed Certificates

HTTPS was configured for the frontend development server using the `@vitejs/plugin-basic-ssl` Vite plugin, which automatically generates a self-signed TLS certificate at build time.

Vite Configuration

The following `vite.config.js` enables HTTPS and proxies API requests to the backend:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import basicSsl from '@vitejs/plugin-basic-ssl'

export default defineConfig({
  plugins: [react(), basicSsl()],
  server: {
    port: 5173,
    https: true,
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true,
      },
    },
  },
})
```

How It Works

1. The `basicSsl()` plugin generates a self-signed certificate and private key on first run.
2. Vite's dev server binds to `https://localhost:5173` using this certificate.
3. API requests to `/api/*` are reverse-proxied to the FastAPI backend at `http://localhost:8000`.
4. The browser shows a certificate warning (expected for self-signed certs), which can be accepted to proceed.

Screenshots

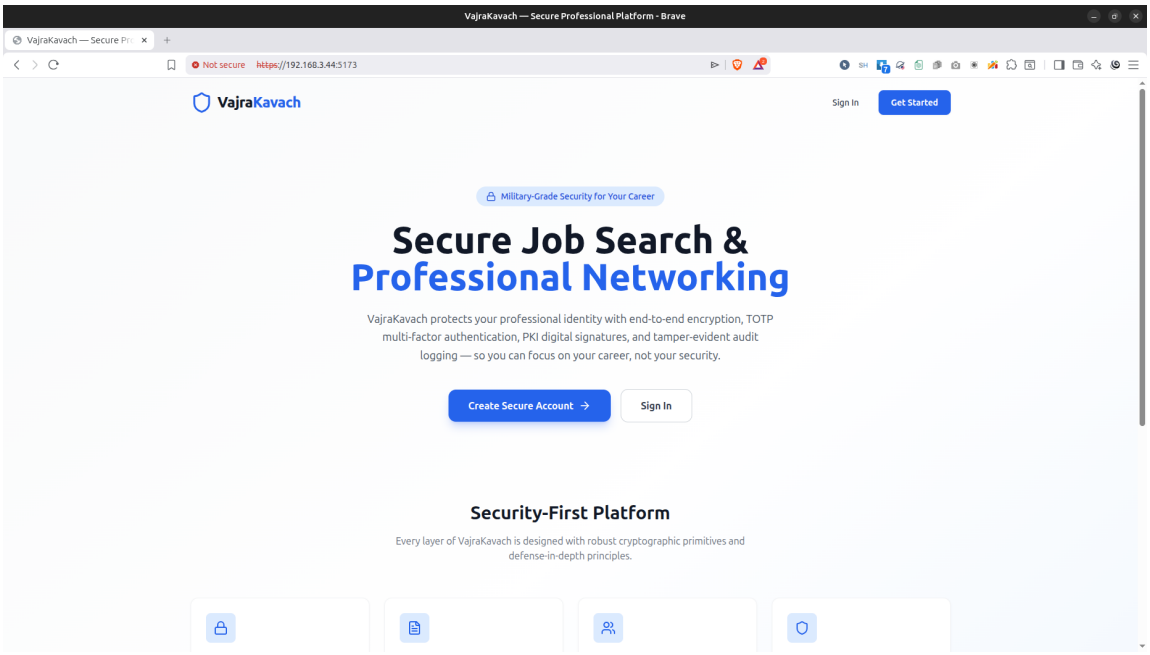


Figure 1: Browser showing HTTPS connection to the application (note the lock icon / certificate warning)

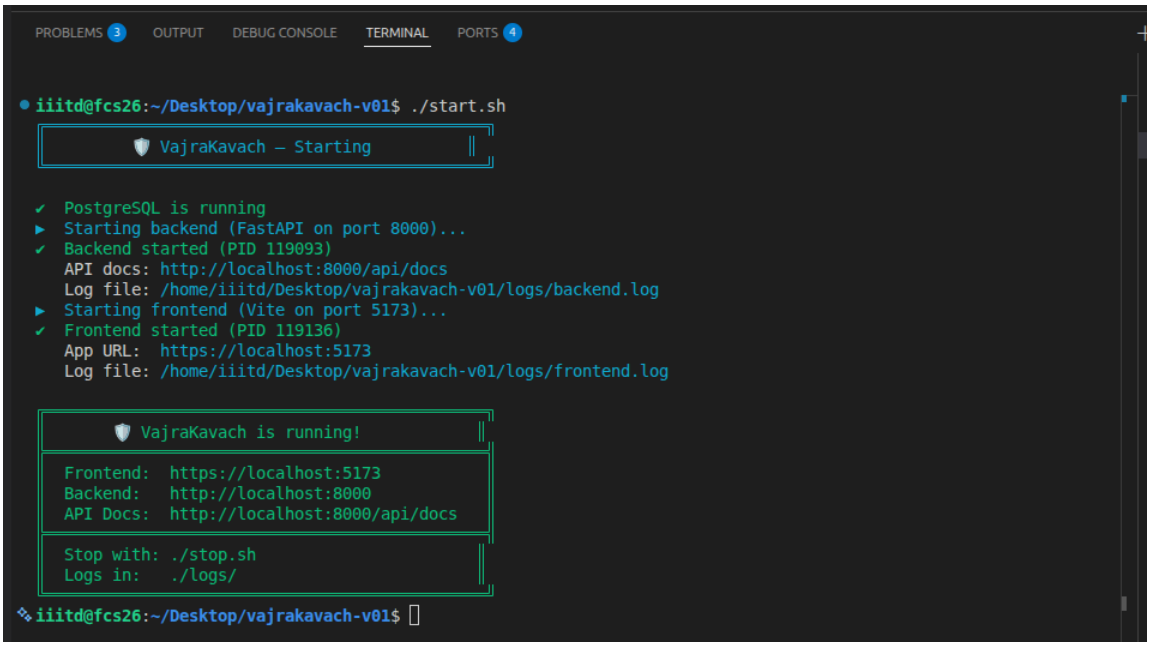


Figure 2: Terminal output showing the server starting with HTTPS on port 5173

Skeleton Application Deployment on VM

The skeleton application was deployed on the IIITD VM with both backend and frontend running simultaneously.

Project Structure

```
vajrakavach-v01/
  backend/
    app/
      main.py            # FastAPI application entry point
      config.py           # Environment configuration
      database.py         # Async SQLAlchemy engine + session
      dependencies.py     # Auth dependency injection
      models/             # SQLAlchemy ORM models
      routes/             # API route handlers
      services/           # Business logic services
    requirements.txt
    venv/
  frontend/
    src/
      pages/              # React page components
      components/         # Reusable UI components
      contexts/           # Auth context provider
      services/           # Axios API client
    vite.config.js
    package.json
  start.sh                # Starts both servers
  stop.sh                 # Stops both servers
  logs/                    # Runtime log files
```

Startup Process

The `start.sh` script automates the full startup:

1. Verifies PostgreSQL is running.
2. Launches the FastAPI backend via Uvicorn on port 8000.
3. Launches the Vite frontend dev server on port 5173 (HTTPS).
4. Displays a summary with all URLs.

```
$ ./start.sh
  PostgreSQL is running
  Backend started (PID ...)
    API docs: http://localhost:8000/api/docs
  Frontend started (PID ...)
    App URL:  https://localhost:5173
```

Screenshots

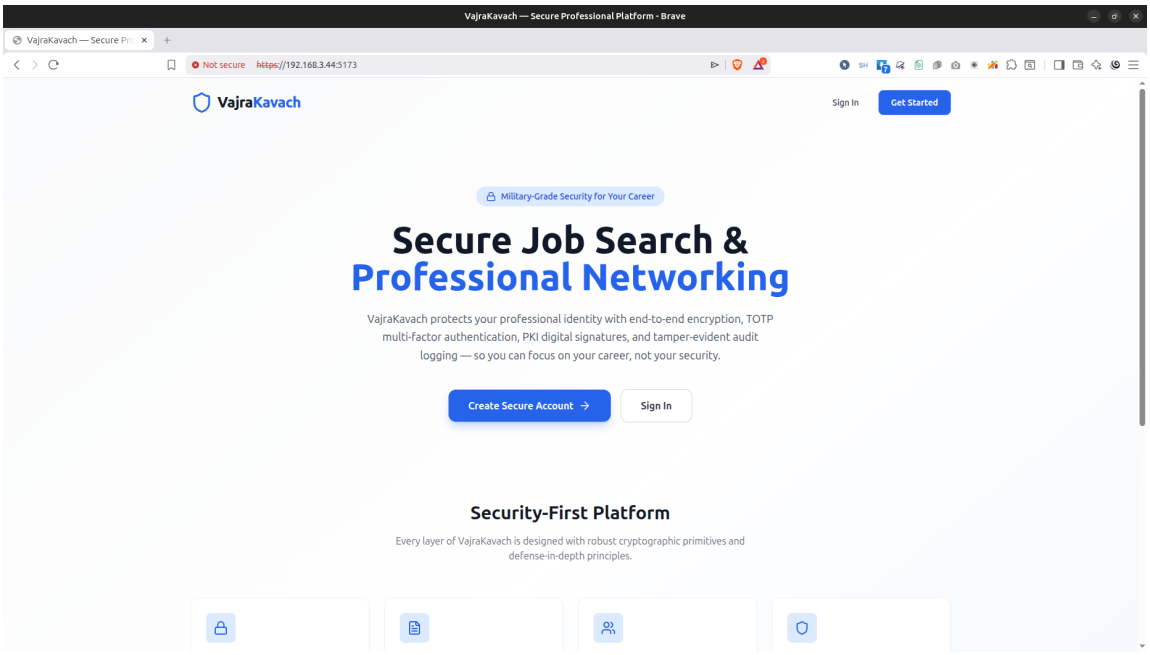


Figure 3: Terminal output of `start.sh` showing both servers launched successfully

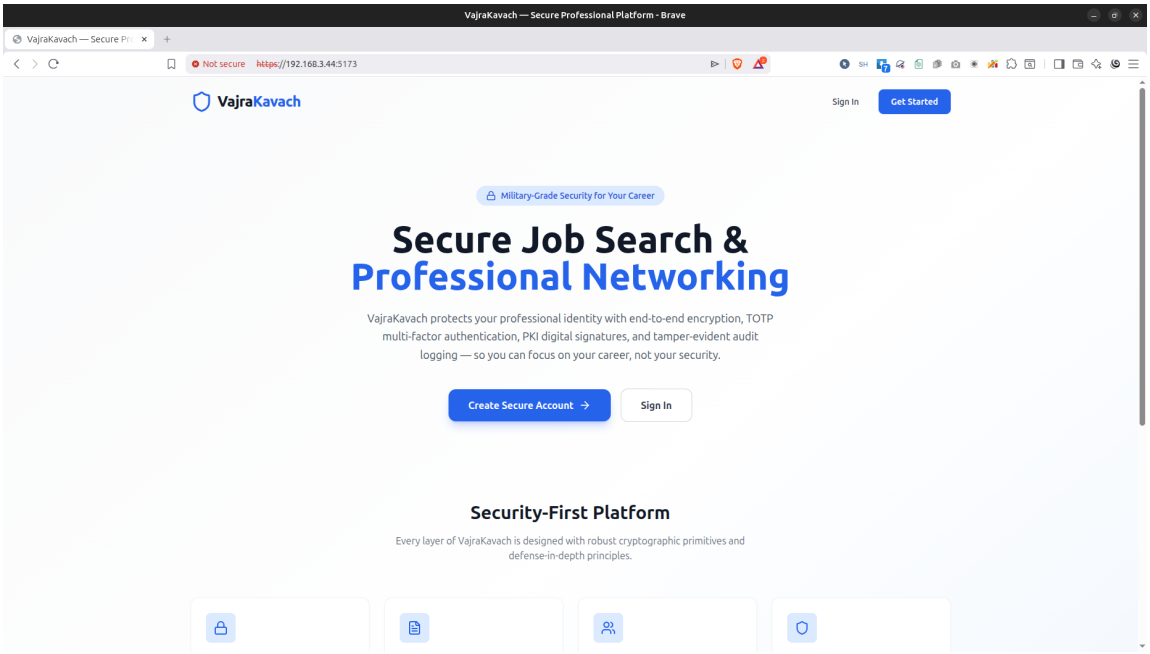
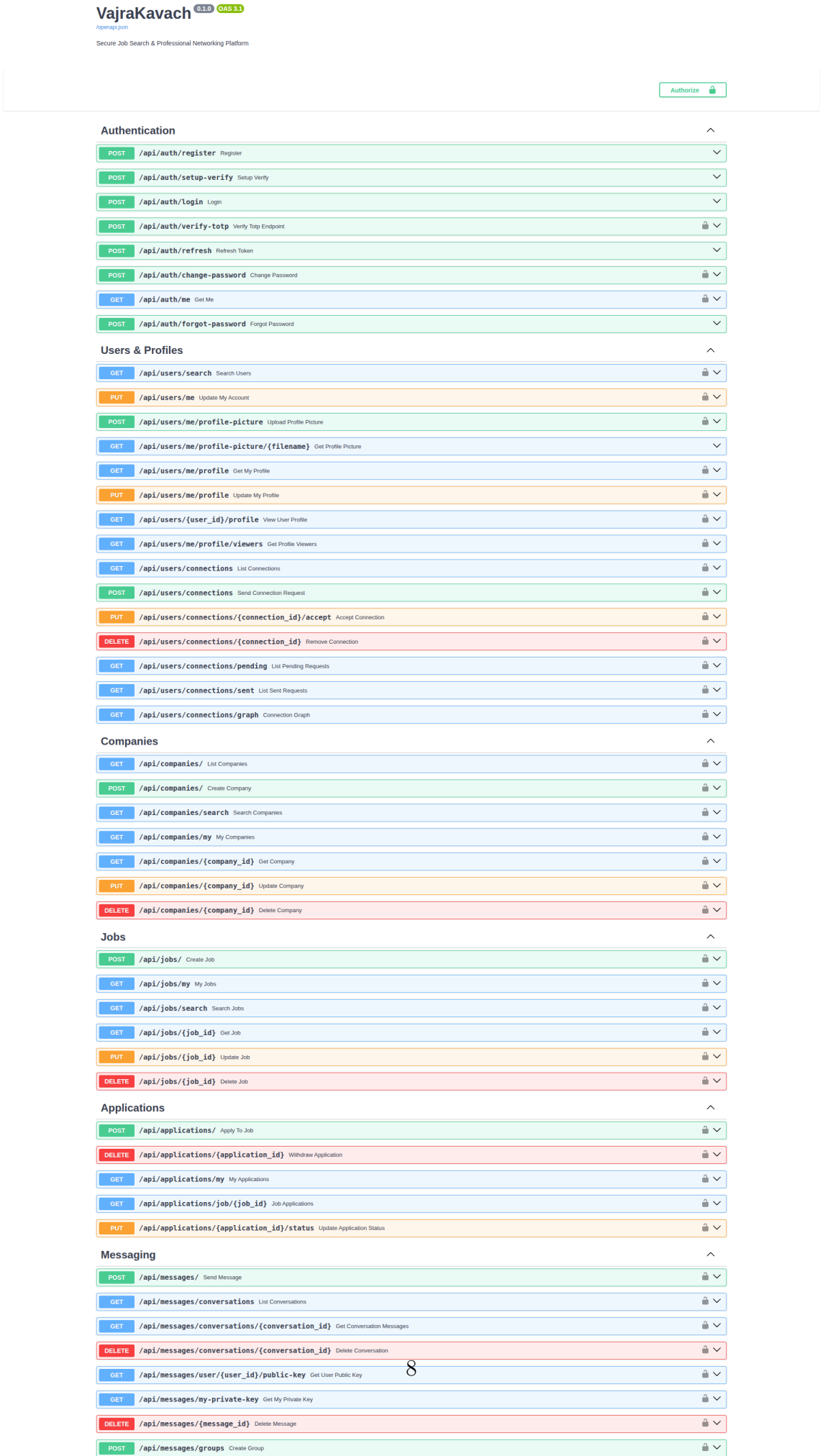


Figure 4: Skeleton application landing page running on the VM


VajraKavach
0.1.0
OAS 3.1
/openapi.json
Secure Job Search & Professional Networking Platform
Authorize
Authentication
POST /api/auth/register Register
POST /api/auth/setup-verify Setup Verify
POST /api/auth/login Login
POST /api/auth/verify-totp Verify Totp Endpoint
POST /api/auth/refresh Refresh Token
POST /api/auth/change-password Change Password
GET /api/auth/me Get Me
POST /api/auth/forgot-password Forgot Password
Users & Profiles
GET /api/users/search Search Users
PUT /api/users/me Update My Account
POST /api/users/me/profile-picture Upload Profile Picture
GET /api/users/me/profile-picture/{filename} Get Profile Picture
GET /api/users/me/profile Get My Profile
PUT /api/users/me/profile Update My Profile
GET /api/users/{user_id}/profile View User Profile
GET /api/users/me/profile/viewers Get Profile Viewers
GET /api/users/connections List Connections
POST /api/users/connections Send Connection Request
PUT /api/users/connections/{connection_id}/accept Accept Connection
DELETE /api/users/connections/{connection_id} Remove Connection
GET /api/users/connections/pending List Pending Requests
GET /api/users/connections/sent List Sent Requests
GET /api/users/connections/graph Connection Graph
Companies
GET /api/companies/ List Companies
POST /api/companies/ Create Company
GET /api/companies/search Search Companies
GET /api/companies/my My Companies
GET /api/companies/{company_id} Get Company
PUT /api/companies/{company_id} Update Company
DELETE /api/companies/{company_id} Delete Company
Jobs
POST /api/jobs/ Create Job
GET /api/jobs/my My Jobs
GET /api/jobs/search Search Jobs
GET /api/jobs/{job_id} Get Job
PUT /api/jobs/{job_id} Update Job
DELETE /api/jobs/{job_id} Delete Job
Applications
POST /api/applications/ Apply To Job
DELETE /api/applications/{application_id} Withdraw Application
GET /api/applications/my My Applications
GET /api/applications/job/{job_id} Job Applications
PUT /api/applications/{application_id}/status Update Application Status
Messaging
POST /api/messages/ Send Message
GET /api/messages/conversations List Conversations
GET /api/messages/conversations/{conversation_id} Get Conversation Messages
DELETE /api/messages/conversations/{conversation_id} Delete Conversation
GET /api/messages/user/{user_id}/public-key Get User Public Key
GET /api/messages/my-private-key Get My Private Key
DELETE /api/messages/{message_id} Delete Message
POST /api/messages/groups Create Group

Summary

For Milestone 1, we have:

1. **Finalized the technology stack** — FastAPI (Python) backend, React (Vite) frontend, PostgreSQL database, with Argon2id password hashing, JWT authentication, TOTP 2FA, Fernet and RSA encryption.
2. **Configured HTTPS** — Self-signed TLS certificates via `@vitejs/plugin-basic-ssl`, serving the frontend over `https://localhost:5173`.
3. **Deployed a skeleton application** — Both backend (port 8000) and frontend (port 5173) running on the IIITD VM with automated startup/shutdown scripts.

The codebase is version-controlled on GitHub and ready for Milestone 2 feature development.

Repository: https://github.com/yashkumarvaibhav/VajraKavach